Acorn Atom+ V191

Acorn Atom+ emulator — ESP32-S3 8048S070 en CYD (V191)

Ik kocht mijn eerste Acorn Atom in 1980 en was meteen enthousiast over de manier waarop je hem zowel in BASIC als in machinetaal kon programmeren. Het was verbazingwekkend wat je ermee kon doen, ondanks het kleine geheugen en de kloksnelheid van 1 MHz. In het pc-tijdperk miste ik het plezier van het programmeren echt. Dat kwam terug met de ESP32 en Arduino-code en zo..... heb ik een praktische **Acorn Atom-emulator voor het ESP32-S3 8048S070 7-inch 800×480-bord** ontwikkeld. Het project richt zich nu op dit platform, inclusief RGB-scherm, SD-opslag, I2S-audio en het optionele fysieke Atom-toetsenbord.

De emulator draait de **originele Acorn Atom 6502-ROM-code en originele Atom-software**. BASIC, de Kernel, cassetteroutines, utility-ROM’s, AtoMMC2 en games worden als echte 6502-instructies in de emulator uitgevoerd; ze zijn niet herschreven als ESP32-applicaties.

De huidige versie is **V191**, de stabiele ontwikkelbasis.



Originele Acorn Atom



Acorn Atom-emulator op de ESP32-S3 8048S070

Belangrijkste kenmerken

- Uitvoering van de originele Acorn Atom BASIC-, Floating Point- en Kernel-ROM’s
- **Fake6502 v1.1**, de MOS 6502-emulatiecore van Mike Chambers
- Atom-compatibele 64 KB-adresruimte en I/O-map
- Regeling van de CPU-snelheid om de nominale **1 MHz** van de Atom te reproduceren, ook wanneer de ESP32 de emulator sneller kan uitvoeren
- Emulatie van de 8255 PPI
- Emulatie van de MC6847-videochip
- Atom-tekst, semigraphics, monochrome graphics en kleurengraphics

- Origineel gedrag van de Atom-toetsenbordmatrix
- Normale ROM-toetsenbordinvoer via serieel, programma's die de matrix rechtstreeks scannen en programma's die direct `FE71` gebruiken
- Afhandeling van SHIFT, CTRL, ESC en BREAK
- Fysiek MCP23017 Atom-toetsenbord op GPIO19/20 met snelle software-I2C-scanning
- SHIFT+BREAK start de geïntegreerde BBC BASIC-omgeving
- Speciale 3D-printbare behuizing voor het reproductie-Atom-toetsenbord
- Ondersteuning voor `.atm`-cassette-images op SD-kaart
- `LOAD` en `SAVE` behouden de voor de Atom zichtbare cassettesemantiek, maar werken veel sneller
- Directory's op SD en automatisch koppelen van `.atm`-bestanden
- **AtoMMC2 / ATMMC3E-ROM op `$E000-$EFFF`**
- AtoMMC2 SD-bestandssysteem met lees- en schrijfmogelijkheid
- Acht omschakelbare **`$A000-$AFFF` utility-ROM-banken**
- **Resident ASBK-S Schakelsoft V24**, vanuit SPIFFS in Atom-RAM geladen
- Automatische initialisatie van Schakelsoft na iedere Atom RESET
- Circulaire zoekfunctie voor utility-ROM-commando's, zodat commando's uit de verschillende banken in willekeurige volgorde gebruikt kunnen worden
- 8048S070-audio via de ingebouwde **MAX98357 I2S-versterker**
- BMP-schermafbeeldingen naar SD-kaart met `~shot`
- Correct cold-startgedrag van Atom BASIC `RND`

ESP32-S3 8048S070-hardware

De huidige emulator is gericht op de **ESP32-S3 8048S070** met het 7-inch 800×480 RGB-scherm. Voor het scherm wordt **LovyanGFX** gebruikt. Het bord levert ook SD-opslag en de MAX98357 I2S-audio-interface.

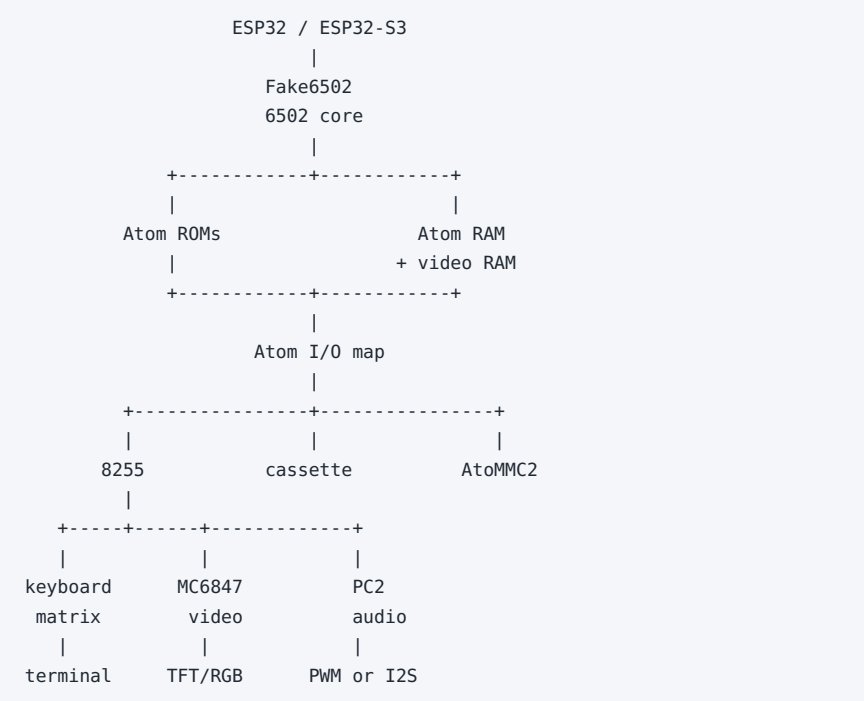
Functie	GPIO
SD SCK	12
SD MISO	13
SD MOSI	11
SD CS	10
MAX98357 BCLK	0
MAX98357 WS/LRCLK	18
MAX98357 DATA	17
Toetsenbord MCP23017 S	DA 19
Toetsenbord MCP23017 S	CL 20

Het fysieke toetsenbord gebruikt **software-I2C**, niet Arduino `Wire`. Eerdere hardware-I2C-proeven gaven stabiliteitsproblemen op dit bord. V187 verving de trage Arduino-`pinMode()`-wisselingen per bit door directe aansturing van de ESP32-S3 GPIO output-enable-registers, met behoud van het bewezen elektrische gedrag: LOW wordt actief gestuurd en HIGH is een werkelijk vrijgegeven/hoogohmige lijn met pull-up. Daarmee steeg de gemeten volledige toetsenbordscan van ongeveer 16 naar circa 76 scans per seconde, zonder de

stabiliteitsregressie van een eerdere permanente `OUTPUT_OPEN_DRAIN`-proef. V188 bevroor deze implementatie als bekende goede toetsenbordbasis en V191 behoudt haar ongewijzigd.

Wat wordt er geëmuleerd?

Dit is geen verzameling ESP32-ports van Atom-games. De ESP32 levert de geëmuleerde hardware, terwijl de originele software op de geëmuleerde 6502 wordt uitgevoerd.



Wanneer een game `$B000-$B003` benadert, het toetsenbord scant, de videomodus wijzigt, PC2 schakelt of een Kernel-routine aanroept, is het de **originele 6502-code** die deze handelingen uitvoert.

CPU-emulatie en snelheidsregeling op 1 MHz

De MOS 6502-uitvoerengine is **Fake6502 v1.1 van Mike Chambers**. Moderne ESP32-hardware kan de Atom sneller emuleren dan de echte machine. Dit is vooral merkbaar op de ESP32-S3: zonder regeling kan de geëmuleerde CPU sneller lopen dan de oorspronkelijke Atom-klok. Daarom gebruikt de emulator een **realtime snelheidsregelaar**. Deze vergelijkt het aantal geëmuleerde 6502-cycli met de verstreken werkelijke tijd en vertraagt de uitvoering waar nodig, zodat de software ongeveer 1.000.000 6502-cycli per seconde ziet.

Dit is belangrijk omdat veel Atom-programma’s uitgaan van een 1 MHz-processor voor:

- software-timinglussen
- games
- toetsenbordscanning
- geluidsopwekking
- cassettetiming

- animatiesnelheid

De regelaar wijzigt het 6502-programma of afzonderlijke instructies niet. De CPU-core kan intern zo snel draaien als de ESP32 toestaat, maar de emulator voorkomt dat de gesimuleerde Atom-tijd sneller voortschrijdt dan op de echte **1 MHz-machine**. Diagnostische uitvoer kan de gemeten effectieve CPU-snelheid tonen, doorgaans dicht bij:

```
[CPU] ... = 1.000 MHz
```

Originele Atom-ROM’s

De normale systeem-ROM’s worden vanuit SPIFFS geladen:

ROM	Adres
`Atom_Basic.rom`	`$C000-$CFFF`
`Atom_FloatingPoint.rom`	`$D000-$DFFF`
`ATMMC3E.rom`	`$E000-$EFFF`
`Atom_Kernel.rom`	`$F000-$FFFF`

Elk ROM-image is 4096 bytes.

Het `$E000`-gebied bevat de **AtoMMC2 ATMMC3E PIC-firmware** in plaats van de Atom DOS-ROM. Dit is de AtoMMC-configuratie voor systemen die een ROM op `$E000` kunnen aanbieden. AtoMMC2 vereist op echte hardware enkele kleine Kernel-aanpassingen; deze zijn daarom ook in deze emulator geïmplementeerd.

De emulator start met:

```
ACORN ATOM+
```

Resident ASBK-S Schakelsoft

V191 laadt het **ASBK-S V24 Schakelsoft**-image vanuit SPIFFS:

```
ASBK-S_v24.bin
```

De binary wordt bij het opstarten naar het geëmuleerde Atom-RAM gekopieerd:

```
$1000-$1660   1633 bytes
$1005         LL97 initialization entry
```

Het wordt bewust als RAM geladen en niet als ROM gemapt. De Atom Kernel moet zijn RESET-verwerking afronden voordat Schakelsoft zijn page-2-vectoren installeert. V191 wacht daarom totdat de 6502 voor het eerst de normale Kernel-`OSRDCH`-entry op `$FE94` bereikt en voert dan het equivalent uit van:

```
JSR $1005
```

LL97 installeert de Schakelsoft-vectoren en status en keert terug naar het onderbroken invoerpad van de Kernel. Dezelfde eenmalige initialisatie wordt na BREAK opnieuw geactiveerd, zodat Schakelsoft na iedere Atom-reset wordt hersteld.

V24 is gebaseerd op de bewezen Schakelsoft-implementatie met circulair zoeken en bewaart daarnaast de AtoMMC-commando- en filing-vectoren die nodig zijn om beide systemen naast elkaar te laten werken. In V191 is gecontroleerd dat Schakelsoft-commandoroutering, AtoMMC `*CAT`/`*CWD`/`*RUN` en de gebankte `$A000` utility-ROM's allemaal samen functioneren.

Copyright van de ROM's

Sommige Acorn- en third-party-ROM-images zijn auteursrechtelijk beschermde software die losstaat van de broncode van deze emulator. Wanneer ROM's niet met de repository worden verspreid, moeten gebruikers de images verkrijgen uit bronnen waarvoor zij gerechtigd zijn ze te gebruiken.

Utility-ROM-banking op `$A000`

Het oorspronkelijke utility-ROM-gebied van de Atom beslaat:

```
$A000-$AFFF
```

De emulator breidt dit uit met een **systeem van acht ROM-banken**, waardoor een nuttige verzameling Atom utility-ROM's naast elkaar kan bestaan.

De huidige banken zijn:

Bank	ROM
0	`Atom_pcharme.rom`
1	`ASBK-I_v1.03.rom`
2	`DISATOM.rom`
3	`SALFAA_2.6.rom`
4	`GAGS_2.3.rom`
5	`PCBOX_v1.02.rom`
6	`TOOLBUG.rom`
7	`AXR-1.rom`

Bij het inschakelen wordt bank 0 geselecteerd.

Het bankmechanisme verandert welk 4 KB-ROM-image voor de 6502 zichtbaar is op `$A000-$AFFF`; de utility-software zelf wordt **niet** gepatcht. Daardoor kunnen originele Atom utility-ROM's worden gebruikt in dezelfde adresruimte waarvoor ze zijn ontworpen.

Schakelsoft-commandoroutering

De originele Atom-firmware verwachtte slechts één utility-ROM op `$A000-$AFFF`. ASBK-S voegt de routering toe die nodig is om de acht banken te laten werken alsof ze één grotere commando-omgeving vormen.

De V24-reconstructie behoudt het historische error-94-overdrachtsmechanisme en voegt een **circulaire zoekactie van één ronde** toe. Als de momenteel geselecteerde utility-ROM een commando niet herkent, zoekt ASBK-S verder door de overige banken, gaat zo nodig van bank 7 terug naar bank 0 en stopt na één volledige ronde.

Als bijvoorbeeld SALFAA in bank 3 geselecteerd is en `SHOW` bij ASBK-I in bank 1 hoort, kan de zoekactie als volgt verlopen:

```
3 -> 4 -> 5 -> 6 -> 7 -> 0 -> 1
```

Onbekende commando's eindigen nog steeds met de normale Atom-fout en veroorzaken geen oneindige lus. ASBK-S levert daarnaast zijn eigen geïntegreerde commando's. De werkende V24-broncode wordt samen met de residente binary in het V191-project bewaard, zodat het gegenereerde image gedocumenteerd en reproduceerbaar blijft.

AtoMMC2-massaopslag

V191 bevat een functionele emulatie van de **AtoMMC2-controllerinterface** samen met de `ATMMC3E` Atom-ROM. AtoMMC2 is ontworpen om de Atom moderne FAT-geformatteerde SD/MMC-opslag te geven. In deze emulator communiceert de originele Atom AtoMMC-ROM met een geëmuleerde controller die rechtstreeks de SD-kaart van de ESP32 gebruikt.

De huidige implementatie ondersteunt de belangrijkste dagelijkse bestandssysteemfuncties, waaronder:

```
*CAT
*CWD
*INFO
*LOAD
*RUN
SAVE
*SAVE
*DELETE
*REN
*MKDIR
*RMDIR
*EXEC
```

Directory-iteratie maakt ook gewone bestanden zichtbaar die voor `*EXEC` nodig zijn. Relatieve en absolute paden worden gecanonicaliseerd en `*`/`?` wildcards worden door de geëmuleerde controllerinterface ondersteund. Bestanden kunnen daardoor vanuit de Atom worden gecatalogiseerd, geladen, uitgevoerd, aangemaakt, hernoemd en verwijderd; ook directory's kunnen worden aangemaakt en doorlopen.

Bijvoorbeeld:

```
*CAT GAMES05
*CWD GAMES05
*CAT
*RUN INVADERSZX
```

`*CAT GAMES05` toont de catalogus van die directory maar maakt deze **niet** de huidige directory. `*CWD GAMES05` wijzigt de huidige AtoMMC-directory. Dit onderscheid is bewust en komt overeen met de AtoMMC-commandosemantiek.

Lange Atom-namen zoals `INVADERSZX` worden ongewijzigd door het AtoMMC-ROM/controllerpad doorgegeven. Zodra met `*CWD` de juiste directory is gekozen, werkt `*RUN INVADERSZX` normaal.

Een BASIC-programma kan normaal worden opgeslagen en opnieuw geladen:

```
10 PRINT "HELLO"
SAVE "TEST"
NEW
LOAD "TEST"
RUN
```

Op de SD-kaart worden Atom-bestanden opgeslagen in `.atm`-containers, terwijl de Atom de normale logische Atom-bestandsnaam ziet.

De AtoMMC2-implementatie bevindt zich bewust op het niveau van de **controllerinterface**, in plaats van AtoMMC-commando's te vervangen door emulator-specifieke equivalenten. De originele `ATMMC3E` 6502-ROM blijft verantwoordelijk voor de verwerking van commando's aan de Atom-zijde.

De dagelijkse AtoMMC2-commandoset werkt in V191. Verdere compatibiliteitswerkzaamheden kunnen nog betrekking hebben op minder vaak gebruikte commando's en random-access-bestandsfunctionaliteit.

MC6847-video-emulatie

De originele Atom gebruikte een Motorola MC6847 Video Display Generator. De emulator interpreteert het Atom-videogeheugen en de relevante VDG-status en rendert het resultaat op het aangesloten TFT/RGB-paneel. De actieve rastergrootte van de Atom wordt behandeld als **256×192**.

Geïmplementeerde ondersteuning omvat:

- Mode 0 alfanumerieke weergave
- Atom-tekenmapping
- het originele Atom-pijltje-omhoogteken
- Mode 0 semigraphics
- monochrome grafische modi
- grafische kleurmodi
- packed-pixel-kleurmodi gebruikt door Floating Point ROM `COLOUR`
- alternatieve MC6847 CSS-kleurset
- videogeheugen vanaf `$8000`
- selectie van de videomodus vanuit de geëmuleerde Atom-I/O-status

Typische monochrome resoluties zijn:

Mode	Raster
1	128×64
2	128×96
3	128×192
4	256×192

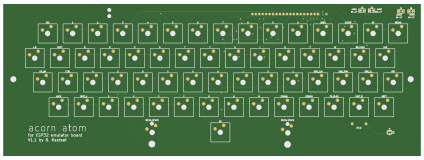
Originele BASIC-grafiekcommando’s en machinetaalgames kunnen daardoor van modus wisselen zonder dat de emulator hoeft te weten welk programma draait.

Fysiek Atom-toetsenbord en toetsenbordemulatie (OPTIONEEL)

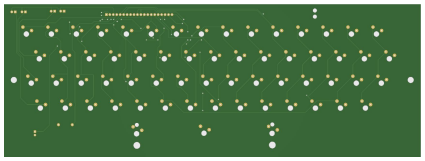
De emulator ondersteunt een **echt Atom-toetsenbord** via een MCP23017 16-bits I/O-expander. Op de 8048S070 zijn hiervoor slechts vier verbindingen nodig: **3V3/VCC, GND, SDA (GPIO19) en SCL (GPIO20)**. De interruptuitgangen van de MCP23017 worden niet gebruikt.

Er zijn drie mogelijke toetsenborden:

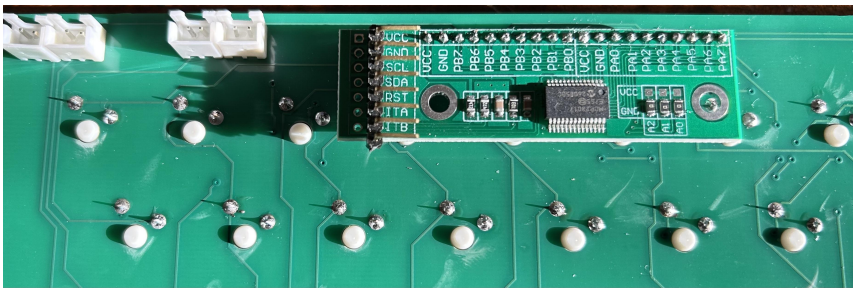
1. Nieuwe Acorn Atom 8×8-toetsenbordprint. Dit is mijn eigen ontwerp met een 8×8-matrix en een directe aansluiting op de MCP23017. Het MCP23017-adres is `0x20`. De productiebestanden, KiCad-ontwerp en Gerbers zijn opgenomen in Acorn Atom toetsenbord — KiCad en Gerbers.



Voorzijde toetsenbordprint



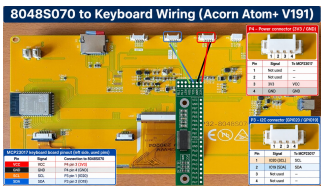
Achterzijde toetsenbordprint



MCP23017-aansluiting op het 8×8-toetsenbord

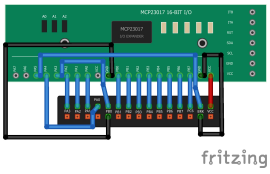


Compleet toetsenbord in de behuizing

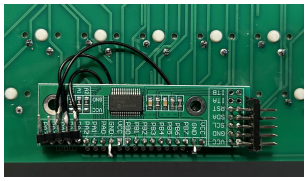


Aansluiting op de 8048S070

2. Herwerkte Acorn Atom-toetsenbordprint van Hackjunk. Zie Hackjunk — Acorn Atom Reworked. Deze print kan via zijn website worden besteld; verzending naar Nederland kostte destijds ongeveer €43 in totaal. De MCP23017 kan niet zonder aanpassingen rechtstreeks op deze print worden gemonteerd. Het MCP23017-adres is **`0x21`**. De foto's hieronder tonen de aansluiting; de foto is gemaakt vóór de adreswijziging van `0x20` naar `0x21`.



MCP23017-bedrading



MCP23017 op de print



Hackjunk-print zonder componenten

3. Klein hulp-toetsenbord met vier toetsen. Dit bevat **CTRL, SHIFT, REPT en BREAK**, plus een aansluiting voor AUX1. Deze toetsen zijn in sommige originele Atom-games nodig en kunnen niet allemaal als afzonderlijke toetsaanslag via een gewone terminal, zoals Tera Term, worden doorgestuurd. Het MCP23017-adres is **`0x22`**.

De firmware herkent het aangesloten toetsenbord aan het MCP23017-I²C-adres. Stel daarvoor de soldeerbruggen **A2, A1 en A0** correct in. De standaardinstelling is `0x20`.

MCP23017-toetsenbordherkenning (V191)

Toetsenbord	MCP23017-adres	A2	A1	A0
Nieuw direct 8×8	`0x20`	GND	GND	GND
Origineel Hackjunk	`0x21`	GND	GND	3,3 V
Klein hulp-toetsenbord	`0x22`	GND	3,3 V	GND

I²C-aansluiting 8048S070: SDA GPIO19, SCL GPIO20, met 3,3V-logica. Een verkeerd adres kan ervoor zorgen dat speciale toetsen verkeerd worden geïnterpreteerd.

Toetsenbordbehuizing

Voor het fysieke Atom-toetsenbord is een speciale **3D-printbare behuizing** ontwikkeld. Deze volgt de verhoudingen en uitstraling van het originele toetsenbord en biedt plaats aan de reproductieprint en MCP23017-interface. Het ontwerp bestaat uit een afzonderlijk deksel en onderstuk, 2,4 mm wanden, PCB-montagepilaren, schuine schroeftorens met verzonken schroefgaten, een kabelopening aan de achterzijde en een LED-opening die met de print is uitgelijnd. De spatiebalk gebruikt een 6.25U-stabilisator. De behuizing is met de toetsenbordprint gepast en is bedoeld voor PLA-printen.

De productie-STL-bestanden zijn bij deze README opgenomen als Atom keyboard V097 PRODUCTION COVER.stl en Atom keyboard V097 PRODUCTION BASE.stl.

De huidige interface koppelt de toetsenbordmatrix aan de MCP23017-GPIO's en scant deze in een taak met lage prioriteit op ESP32-S3 Core 0. V191 behoudt het bewezen held-key-gedrag en gebruikt de snellere true-release software-I2C-implementatie die in V187 is ontwikkeld en in V188 als bekende goede basis is bevroren. Korte fysieke toetsaanslagen worden bovendien maximaal 750 ms als eenmalige

gebeurtenis bewaard voor een externe `$FE71`-aanroep. Daardoor kan een korte echte toetsaanslag niet meer verdwijnen tussen twee aanroepen van de Kernel-toetsenbordscanner. Seriële invoer behoudt de bestaande FIFO/FE71-latch, terwijl directe/raw-matrixlezers uitsluitend de werkelijk actuele fysieke toetsstatus zien. Deze generieke wijziging herstelde een snelle en betrouwbare bediening via zowel fysiek toetsenbord als serieel in originele software zoals Frogger, zonder game-specifieke patch of timingwijziging.

Originele Atom-software leest het toetsenbord op verschillende manieren. Normale BASIC/ROM-invoer gebruikt de toetsenbord-FIFO; games die rechtstreeks de 8255 scannen zien een vastgehouden fysieke toetsmatrix; en software die de Kernel- scanner op `$FE71` aanroept heeft een apart pad voor afzonderlijke toetsaanslagen. Dit is belangrijk voor de compatibiliteit met originele games.

De V097-behuizing is ontworpen voor de nieuwe 8×8-print. De oorspronkelijke Hackjunk-print wijkt horizontaal ongeveer **3 mm** af en past daarom niet zonder aanpassingen. Voor montage van de Hackjunk-print moeten nieuwe PCB-bevestigingsgaten circa 3 mm links van de bestaande gaten worden geboord en moet de printrand op enkele plaatsen voorzichtig worden bijgevijld om ruimte te maken voor de bevestigingspilaren. **De nieuwe 8×8-print past zonder deze wijzigingen in de V097-behuizing.**

In de steunpilaren van de behuizingsbodem kunnen **Ruthex M3×5,7 mm messing schroefdraadinserts** worden aangebracht: Ruthex M3×5,7 draadinserts.



M3-draadinserts



Insert geplaatst



Insert verwarmen



Definitieve montage

Houd de punt van een soldeerbout tegen de bovenzijde van de insert om deze voorzichtig in het PLA te laten zakken. De print kan daarna met een bout van 10 mm worden vastgezet; ik gebruikte een extra ring en moer om de hoogte passend te maken. Het **deksel** kan worden bevestigd met M3×10 mm bouten aan de voorzijde en M3×25 mm bouten aan de achterzijde.

BREAK en SHIFT+BREAK

De fysieke **BREAK**-toets voert een Atom-reset uit en wist tijdelijke toetsenbord- en cassettestatus voordat de normale Atom-start wordt uitgevoerd. ASBK-S V24 wordt daarna op het juiste moment in de Kernel-start opnieuw geïnstalleerd.

SHIFT+BREAK kiest de BBC BASIC-omgeving. Eerst vindt een normale Atom-reset plaats, ASBK-S-initialisatie wordt voor deze start overgeslagen en daarna wordt BBC BASIC gestart via het equivalent van `LINK #4000`. Het commando wordt één teken per echte OSRDCH-aanroep aangeboden, omdat dit met de timing van de originele ROM

de betrouwbare methode bleek. Bij binnenkomst wordt het scherm gewist en de Atom-cursor naar linksboven gezet zodat BBC BASIC bovenaan het scherm begint.

Cassette- en `.atm`-ondersteuning

De emulator behoudt ook zijn cassettegeoriënteerde opslagondersteuning.

De standaard softwaredirectory is:

```
/ATOM/TAPES
```

ATM-metadata bevat:

- Atom-bestandsnaam
- laadadres
- uitvoeradres
- datalengte
- programmadata

De emulator kan de Atom-cassetteblokstroom opbouwen, inclusief headers en checksums. Turbopaden versnellen veelgebruikte `LOAD`- en `SAVE`-bewerkingen, terwijl de voor de originele ROM zichtbare Atom-semantiek behouden blijft. Dit cassettepad blijft onafhankelijk van AtoMMC2 bruikbaar.

Audio

De geluidsbron van de Atom

De Atom heeft geen aparte geluidssynthesizer. De luidspreker wordt bestuurd door **8255 Port C bit 2 (PC2)**. Originele software maakt geluid door PC2 zelf te veranderen. De timing en het patroon van die overgangen maken daarom deel uit van het oorspronkelijke programma.

De emulator volgt hetzelfde principe:

```
original 6502 program
        |
    writes PC2
        |
   emulated 8255
        |
 timestamped PC2 edges
        |
 audio reconstruction
        |
    speaker output
```

Er zijn geen vooraf opgenomen effecten en geen gamespecifieke geluidstabellen.

8048S070-audio: doorlopende 6502-cyclustijdlijn

De 8048S070 gebruikt zijn **MAX98357 I2S-versterker**. De huidige geluidsrenderer registreert iedere PC2-overgang met het **absolute geëmuleerde 6502-cyclusnummer**. De audiotaak houdt een doorlopende 6502-tijdcursor bij en reconstrueert de resulterende golfvorm op **80 kHz**.

Dit is belangrijk omdat Atom-software PC2 zeer snel kan schakelen. De audiorenderer volgt daarom de timing die door de originele 6502-code wordt gegenereerd, in plaats van bij iedere PC2-wijziging een synthetisch geluid te maken. Een eenvoudig AC-koppel/hoogdoorlaatmodel past de digitale PC2-golfvorm aan tot een luidsprekerachtig signaal voordat dit naar I2S wordt gestuurd.

Deze generieke methode heeft verschillende originele effecten succesvol gereproduceerd, zoals:

- het marcheren van aliens
- schieten
- verschillende vernietigingsgeluiden
- continue effecten
- speler-/geschuteffecten

De exacte klankkleur hoeft elektrisch niet identiek te zijn aan een veertig jaar oud Atom-luidsprekercircuit, maar de **brongolfvorm en timing komen uit het originele Atom-programma**.

Cold-startgedrag van BASIC `RND`

Atom BASIC bewaart zijn vijf bytes tellende randomstatus op:

```
$0008-$000C
```

Als al het geëmuleerde RAM bij het opstarten op nul wordt gezet, blijven deze bytes in een volledig nulstatus staan waaruit het originele BASIC-algoritme voor willekeurige getallen niet kan herstellen. Een echte Atom garandeert geen met nul gevuld RAM bij inschakelen. Daarom initialiseert de emulator deze vijf bytes met een deterministische niet-nulwaarde:

```
78 56 34 12 90
```

Daarna laat de **originele BASIC-ROM** de randomgenerator normaal verder lopen. Er wordt geen vervangende `RND`-implementatie gebruikt. Dit is opnieuw een voorbeeld van de ontwerpregel van het project: corrigeer de toestand van de geëmuleerde machine in plaats van de applicatie te patchen.

Seriële diagnostiek en spiegeling van Atom-uitvoer

V191 houdt de normale seriële uitvoer relatief rustig, maar behoudt bovenaan de hoofdsketch schakelaars voor diagnostiek:

```
#define SERIAL_SHOW_ATOMMC_COMMENTS      0
#define SERIAL_SHOW_CPU_DIAGNOSTIC       0
#define SERIAL_SHOW_RX_MATRIX_DIAGNOSTIC 0
```

Zet een schakelaar op `1` wanneer die diagnostiek nodig is.

Een aparte, vanuit de Atom zichtbare instelling spiegelt echte Atom-schermuitvoer naar de seriële poort van de ESP32:

```
?#BFFE=1
```

schakelt de spiegeling in, en:

```
?#BFFE=0
```

schakelt deze uit. De spiegeling gebruikt de originele teken-/controlestroom vóór de omzetting naar MC6847-schermcodes. Daardoor is dit bruikbaar voor assemblerlistings en diagnostiek zonder de eigen uitvoerroutines van de Atom te vervangen.

Schermafbeeldingen

De emulator kan het huidige scherm als ongecomprimeerde **24-bit BMP** op de SD-kaart opslaan.

Schermafbeeldingen worden opgeslagen onder:

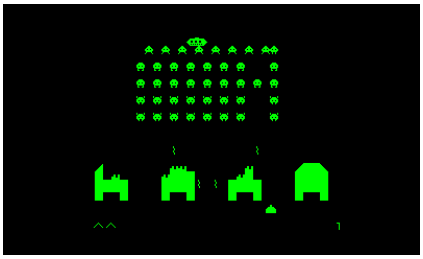
```
/ATOM/SCREENSHOTS
```

met automatisch gegenereerde bestandsnamen zoals:

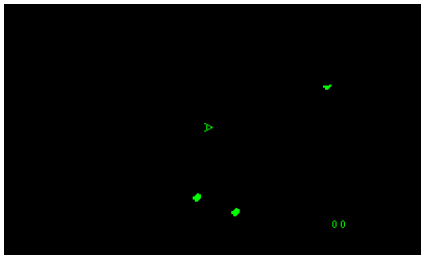
```
ATOM0001.BMP
ATOM0002.BMP
```

Voorbeelden van programma’s die op de emulator draaien

De volgende afbeeldingen zijn directe schermafbeeldingen van het geëmuleerde Atom-scherm. Ze tonen verschillende soorten software die op V191 draaien: games, utility-ROM-software, graphics en de normale Atom-omgeving.



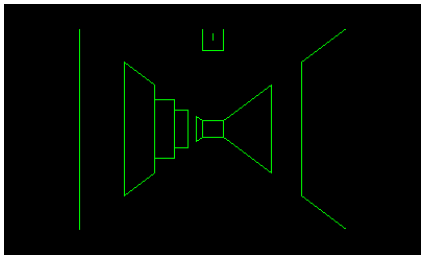
Programmavoorbeeld — Invaders uitgevoerd als 6502-code.



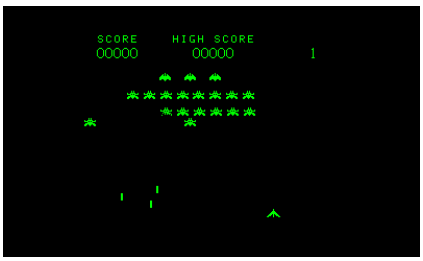
Programmavoorbeeld — Asteroids, origineel Atom-programma op het geëmuleerde scherm.



Utility-ROM-software — PCBOX, gebankte utilities beschikbaar op $A000.



Grafisch voorbeeld — Minotaur met Atom-graphics gerenderd door de MC6847-emulatie.



Programmavoorbeeld — Galaxian, origineel Atom-programma.



Programmavoorbeeld — Frogger, origineel Atom-kleurenspel.

Indeling van de SD-kaart

Een typische SD-kaart kan het volgende bevatten:

```
/ATOM
+-- TAPES
|   +-- FROGGER.atm
|   +-- REVERSI.atm
|   +-- BREAKOUT.atm
|   +-- Games01
|   |   +-- ...
|   |   `-- ...
|   `-- ...
`-- SCREENSHOTS
    +-- ATOM0001.BMP
    `-- ...
```

AtoMMC2 kan rechtstreeks door het op SD gebaseerde bestandssysteem navigeren, terwijl de oorspronkelijke cassette/ATM-interface beschikbaar blijft.

De Acorn Atom+ bouwen

Het project kan worden gebouwd met de **ESP32-S3 8048S070** (het hoofdmodel met 7-inch scherm) of de **JC2432W328C CYD** (de alternatieve uitvoering). Een fysiek Atom-toetsenbord is optioneel. De KiCad-bestanden en Gerbers staan in `keyboard/Acorn_Atom_Kicad_Production.zip`; de 3D-printbare toetsenbordbehuizing staat in `keyboard/`.

U kunt de Gerber-bestanden naar een PCB-fabrikant sturen, bijvoorbeeld via de onderstaande link. Ik heb zelf nog enkele printplaten beschikbaar; vraag er gerust naar.

Onderdelen en leveranciers

Affiliateverklaring: Sommige onderstaande links zijn affiliatelinks. Als u via deze links iets koopt, kan ik een commissie ontvangen zonder extra kosten voor u.

Onderdeel	Doel	Besteladres
8048S070	Hoofdprocessor en 7-inch scherm voor de Acorn Atom+	Amazon – 8048S070
Keycaps	Toetsdoppen voor het Atom-toetsenbord; controleer indeling en toetsformaten	Amazon – keycaps
MX2-compatibele mechanische schakelaars	Schakelaars voor de toetsenbordprint	Amazon – MX2-schakelaars
6,25U-spatiebalkstabilisator	Stabilisatie van de lange spatiebalk	Amazon – 6,25U-stabilisator
Toetsenbordprint (PCB)	Productie met de meegeleverde KiCad- en Gerber-bestanden	JLCPCB – PCB bestellen
MCP23017	Breidt de I²C-aansluiting uit tot 16 I/O-lijnen voor het toetsenbord	Amazon – MCP23017
JC2432W328C CYD	Display/controller voor de alternatieve CYD-uitvoering	Amazon – JC2432W328C

Verder zijn geschikte connectoren, bedrading en de onderdelen voor de montage van de print en behuizing nodig. Controleer altijd de bordrevisie en pinbezetting voordat u de voeding aansluit.

Montage in het kort

1. Bestel de toetsenbordprint met de Gerbers uit `keyboard/` of gebruik een compatibel Atom-toetsenbord.
2. Monteer de MX-schakelaars, keycaps, 6,25U-stabilisator en de benodigde interfaceonderdelen.
3. Print indien gewenst de meegeleverde onder- en bovenkant van de behuizing.
4. Sluit de MCP23017 aan volgens het bedradingsschema van het **gekozen displaybord**. Beide versies gebruiken 3,3V-logica: sluit VCC van het toetsenbord **niet op 5 V** aan.
5. Compileer en upload de bijbehorende V191 Arduino-sketch. Bereid de vereiste ROM-bestanden lokaal voor volgens de instructies hieronder.

Software compileren

Het project gebruikt de **Arduino ESP32-core**. De schermbibliotheek hangt af van het doelbord:

- ESP32-S3 8048S070: **LovyanGFX** van Lovyan03/tobozo V1.2.27

De 8048S070-build gebruikt daarnaast de ESP32-I2S-faciliteiten voor de ingebouwde MAX98357-audio-uitgang. Seriële diagnostiek gebruikt:

```
115200 baud
```

ROM-images moeten in SPIFFS worden geïnstalleerd met de bestandsnamen die de sketch verwacht. V191 vereist ook het residente Schakelsoft-image:

```
ASBK-S_v24.bin
```

Een deel van de ROM-binaries wordt niet met het project meegeleverd; volg daarvoor **De V191 ROM-`data`-directory voorbereiden** hieronder. Het meegeleverde Python-hulpprogramma identificeert gedownloade Atom-ROM's via hun checksum, verwerkt de ondersteunde historische archiefformaten en bouwt een gecontroleerde `data_ready`-directory. De resulterende V191-`data`-directory moet naar SPIFFS worden geüpload na wijzigingen aan de ROM-set of de ASBK-S binary.

De V191 ROM-`data`-directory voorbereiden

V191 gebruikt originele Acorn Atom- en third-party-ROM-software. Om de installatie eenvoudig te maken bevat V191 het Python-hulpprogramma:

```
prepare_atom_roms_FINAL_v12.py
```

Het hulpprogramma identificeert ROM's aan de hand van hun **historische Acorn Atom-ROM-checksum**, niet aan de hand van de bestandsnaam. Dit is belangrijk omdat dezelfde ROM vaak onder verschillende bestandsnamen en in verschillende oude Atom-archief-/containerformaten voorkomt.

Downloadbronnen

Maak een directory met bijvoorbeeld de naam `Roms` en download daarin de onderstaande bronbestanden of archieven. Normaal gesproken is het niet nodig de ROM's vóór het uitvoeren van het voorbereidingsprogramma te hernoemen, te strippen of handmatig uit te pakken.

Atomulator-ROM's

Download het Atomulator-masterarchief:
https://github.com/hoglet67/Atomulator/archive/refs/heads/master.zip
Dit archief bevat de BASIC-, Floating Point- en Kernel-ROM's. Het voorbereidingsprogramma vindt de benodigde ROM's automatisch.

V191-bestand	Atom-checksum	Bron
`Atom_Basic.rom`	`D67D`	Atomulator-directory `roms`
`Atom_FloatingPoint.rom`	`AAA1`	Atomulator-directory `roms`

V191-bestand	Atom-checksum	Bron
`Atom_Kernel.rom`	`F0B9`	Atomulator Kernel, automatisch gepatcht voor AtoMMC2

Wouter Ras Atom-ROM-collectie

Download `atom.zip`:
https://www.stairwaytohell.com/atom/wouterras/atom.zip
Dit archief bevat verschillende benodigde ROM's, waaronder P-Charme, SALFAA, GAGS en AXR-1. Het voorbereidingsprogramma zoekt en converteert de benodigde images automatisch.

V191-bestand	Atom-checksum	Bron
`Atom_pcharme.rom`	`72FD`	door Frans van Hoesel: `atom.zip`, in `ExtROM.Tap`
`SALFAA_2.6.rom`	`3626`	door B. Poot en later uitgebreid door J. Jobse: `atom.zip`; archiefimage `9F75` wordt automatisch geconverteerd
`GAGS_2.3.rom`	`19C1`	door Gerrit Hillebrandt: `atom.zip`; archiefimage `196E` wordt automatisch genormaliseerd
`AXR-1.rom`	`CCEB`	door Jos Horsmeijer: `atom.zip`, in `ExtROM.Tap`

DISATOM

Download `disatom`:
https://www.acornatom.nl/atomarch/acl1-31/disatom

V191-bestand	Atom-checksum	Bron
`DISATOM.rom`	`7049`	door J.R. Stevenson & J.C. Rocket: Acorn Atom-archief `acl1-31`

PCBOX

Download `pcbox.rom`:
https://www.acornatom.nl/atomarch/atominpc.108/pcbox.rom

V191-bestand	Atom-checksum	Bron
`PCBOX_v1.02.rom`	`41CE`	door Roland Leurs: Atom-in-PC 1.08-archief

TOOLBUG

Download `toolbug`:
https://www.acornatom.nl/atomarch/acl2-05/toolbug

V191-bestand	Atom-checksum	Bron
`TOOLBUG.rom`	`129F`	door Psion: Acorn Atom-archief `acl2-05`

BBC BASIC

Download het 16KB-bestand `ATBASIC3` (BBC BASIC voor de Acorn Atom, J. G. Harston). Het voorbereidingsscript controleert de SHA-256-checksum:

```
a7310823149731872e326dbe6a0d56769e0435390465bc872b2e4e9aa0c05c73
```

Met het project meegeleverde ROM's

De drie ROM-images waarvoor distributietoestemming is verleend, worden met V191 meegeleverd. Ze hoeven niet afzonderlijk te worden gedownload.

V191-bestand	Atom-checksum	Herkomst
`ASBK-I_v1.03.rom`	`988C`	Aren Slootweg en Bas Kasteel; V1.03
`ASBK-S_v24.bin`	`57CD`	Aren Slootweg en Bas Kasteel; V24, exact 1633 bytes
`ATMMC3E.rom`	`5072`	AtoMMC v3.02, **PIC / E000**-versie; met toestemming van David Banks

Gebruik de **PIC E000**-versie van AtoMMC v3.02 en niet de AVR-versie met een vergelijkbare naam. De binaire inhoud verschilt. De toestemming voor het meeleveren van deze ROM is verleend door David Banks.

Het voorbereidingsprogramma uitvoeren

Het script `prepare_atom_roms_FINAL_v12.py` staat in `Roms`. Plaats het samen met de gedownloade bestanden in de directory `Roms` samen met de gedownloade bestanden en archieven en voer uit:

```
python prepare_atom_roms_FINAL_v12.py
```

Het programma scant de directory recursief. Het begrijpt de formaten die in de historische Atom-archieven voorkomen, waaronder:

- gewone ROM-images van 4096 bytes;
- 16 KiB CDEF-ROM-collecties;
- ZIP-archieven;
- multi-ROM `.Tap`-bestanden zoals `ExtROM.Tap`;
- Atom-PC-wrappers van 4112 en 4118 bytes;
- Atom-archiefbestanden van 4351 bytes.

Het maakt:

```
Roms/
`-- data_ready/
```

Kies ROM's **niet** alleen omdat hun bestandsnamen correct lijken. Het voorbereidingsprogramma controleert de werkelijke binaire inhoud.

Acorn Atom-checksum

Het voorbereidingsprogramma gebruikt de historische **Acorn Atom-ROM-checksum / signature** waarmee Atom-ROM-images worden geïdentificeerd. Voor de ROM-identificatie wordt bewust niet vertrouwd op CRC32, MD5, SHA-1 of SHA-256.

Nuttige referentiewaarden zijn:

```
D67D  original Acorn Atom BASIC
AAA1  original Acorn Atom Floating Point
E386  original Acorn Atom Kernel
```

De V191-Kernel wijkt bewust af van de originele Kernel vanwege de AtoMMC2 E000-integratie. De vereiste uiteindelijke checksum is `F0B9`.

Waar nodig verwijdert het hulpprogramma eerst een herkend archief-/headerformaat en berekent vervolgens de Atom-checksum over de werkelijke ROM-payload. Daarna:

1. identificeert het bekende ROM-versies via hun checksum;
2. voert het alleen conversies uit die expliciet voor V191 zijn geverifieerd;
3. geeft het resultaat de bestandsnaam die de emulator verwacht;
4. schrijft het bestand naar `data_ready`;
5. controleert het de complete V191-set opnieuw.

Zo bevat de Wouter Ras `ExtROM.Tap` archiefvarianten van GAGS en SALFAA. Deze worden aan hun eigen checksums herkend en geconverteerd naar de exacte images die V191 nodig heeft:

```
GAGS 2.3
196E -> 19C1 -> GAGS_2.3.rom
SALFAA 2.6
9F75 -> 3626 -> SALFAA_2.6.rom
```

Onbekende ROM’s worden gemeld maar niet stilzwijgend gebruikt als vervanging voor een vereiste V191-ROM.

Vereiste uiteindelijke set

Een geslaagde voorbereiding moet alle **13/13** bestanden als `OK` melden:

```
D67D  Atom_Basic.rom
AAA1  Atom_FloatingPoint.rom
5072  ATMMC3E.rom
F0B9  Atom_Kernel.rom
72FD  Atom_pcharme.rom
988C  ASBK-I_v1.03.rom
7049  DISATOM.rom
3626  SALFAA_2.6.rom
19C1  GAGS_2.3.rom
41CE  PCBOX_v1.02.rom
129F  TOOLBUG.rom
CCEB  AXR-1.rom
57CD  ASBK-S_v24.bin       1633 bytes   (project-supplied V24 image)
```

Aan het einde van het rapport moet daarom staan:

```
RESULT: COMPLETE
```

Als een regel met `X` is gemarkeerd, verkrijg dan de ontbrekende/juiste bron-ROM en voer het hulpprogramma opnieuw uit. Kopieer niet zomaar een onbekende ROM naar `data_ready` om alleen aan de bestandsnaam te voldoen.

De voorbereide bestanden installeren

Zodra het rapport alle 13 regels als `OK` toont, kopieer je de inhoud van `data_ready` naar de `data`-directory van het Arduino-project. Die `data`-directory wordt vervolgens met de normale filesystem-uploadprocedure voor de gekozen Arduino/ESP32-omgeving naar het SPIFFS-bestandssysteem van de ESP32 geüpload.

De resulterende SPIFFS-dataset levert de vier systeem-ROM’s, acht gebankte utility-ROM’s en de residente ASBK-S-binary die V191 nodig heeft.

Aan de slag

Na een succesvolle start moet het scherm tonen:

```
ACORN ATOM+

>
```

Schakelsoft is op dat moment al resident en geïnitialiseerd. Utility-ROM-commando’s kunnen direct worden ingevoerd; een ASBK-S `*EXEC` of handmatige `LINK #1005` is niet nodig.

AtoMMC2 kan vervolgens rechtstreeks worden gebruikt:

```
*CAT
```

Ga naar een directory:

```
*CWD Games01
*CAT
```

Laad en start software:

```
*LOAD FROGGER
*RUN FROGGER
```

of gebruik waar van toepassing de traditionele BASIC-/cassettecommando’s van de Atom:

```
LOAD "FROGGER"
RUN
```

V191-compatibiliteitsmijlpaal

V191 is de huidige bekende goede mijlpaal. De bewezen PIC ATMMC3E-AtoMMC2- omgeving, residente ASBK-S V24, acht gebankte `$A000` utility-ROM’s, BBC BASIC via SHIFT+BREAK en de in V188 bevroren

toetsenbordbasis blijven behouden. De aanvullende toetsenbordverbetering in V191 voegt een generieke eenmalige fysieke toetsgebeurtenis toe voor externe `$FE71`-aanroepen. Diagnostiek liet zien dat sommige originele software FE71 zo weinig frequent opvraagt dat een korte fysieke toetsaanslag volledig tussen twee aanroepen kan vallen; seriële invoer verloor dezelfde aanslag niet omdat die in de FIFO bleef staan. V191 geeft fysieke FE71-invoer dezelfde betrouwbaarheid als afzonderlijke gebeurtenis, zonder het normale held-key- of raw-matrixgedrag te veranderen. Daarna zijn zowel het fysieke toetsenbord als seriële bediening met het originele Frogger- programma correct getest, zonder Frogger-specifieke wijziging.

Bevestigde functionaliteit omvat:

- Atom start naar BASIC
- uitvoering van de originele BASIC-, Floating Point- en Kernel-ROM’s
- gereguleerde 6502-uitvoering op ongeveer 1 MHz
- normale BASIC-toetsenbordinvoer
- gamebesturing via rechtstreekse matrixscanning
- directe `FE71`-toetsenbordgebruikers
- BREAK-afhandeling
- MC6847-tekst en graphics
- monochrome en kleurengraphics
- toegang tot SD-kaart
- laden en opslaan van `.atm`-cassettes
- SD-directorynavigatie
- schermafbeeldingen
- achtbank `$A000` utility-ROM-systeem
- ASBK-I v1.03 in utility-ROM-bank 1
- residente ASBK-S V24 op `$1000-$1660`
- automatische ASBK-S-initialisatie op `$1005` na RESET
- circulaire Schakelsoft-commandoroutering over alle acht utility-ROM-banken
- `ATMMC3E` op `$E000`
- `ACORN ATOM+` bij opstarten
- AtoMMC2-catalogus en directorynavigatie
- AtoMMC2 load/run
- AtoMMC2 save/write
- verwijderen en hernoemen
- directory’s aanmaken/verwijderen
- AtoMMC2 `*EXEC`, wildcardmatching en verwerking van absolute/relatieve paden
- continue PC2/I2S-audio op 8048S070
- correctie van het cold-startgedrag van BASIC `RND`

Tijdens de ontwikkeling is een aanzienlijke verzameling originele Atom-software gebruikt. Compatibiliteitswerk blijft gericht op het reproduceren van de machine-interface in plaats van het herkennen van afzonderlijke applicaties.

Ontwerpfilosofie

De terugkerende regel is:

> **Herken geen game om vervolgens het resultaat na te bootsen. Emuleer het Atom-mechanisme dat de game verwacht.**

Voorbeelden:

- graphics worden gekozen op basis van Atom/MC6847-status, niet op basis van de naam van een game;
- geluid komt van echte, van tijdstempels voorziene PC2-overgangen, niet van vooraf opgenomen effecten;
- de CPU-snelheid wordt globaal geregeld in plaats van timinghacks aan afzonderlijke programma's toe te voegen;
- `RND` is gecorrigeerd door de cold-startmachinestatus te herstellen;
- toetsenbordcompatibiliteit onderscheidt de verschillende manieren waarop echte Atom-software het toetsenbord leest;
- AtoMMC2 draait zijn originele `$E000`-ROM tegen een geëmuleerde controllerinterface;
- `$A000`-banking maakt echte utility-ROM-images zichtbaar in plaats van hun commando's opnieuw te implementeren;
- `.atm`-afhandeling behoudt Atom-bestandsnamen, adressen en programmadata.

Daardoor kan één correctie de compatibiliteit van veel programma's verbeteren.

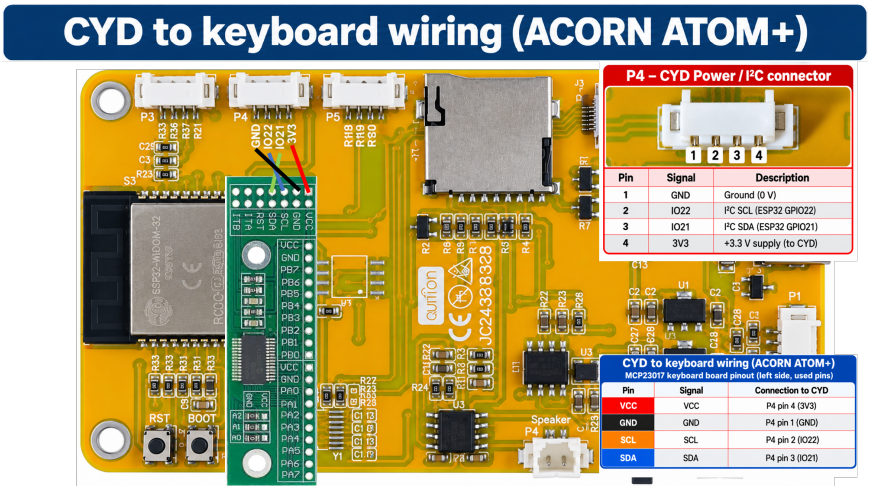
Alternatieve hardware: JC2432W328C CYD

De repository bevat ook **`ATOM_6502_CYD_V191`**, een uitvoering van de Atom-emulator voor de CYD. De geëmuleerde Atom en de toetsenbordinterface werken volgens hetzelfde principe als op de 8048S070, maar de schermhardware, GPIO-toewijzingen en geluidsuitgang verschillen. Gebruik voor dit bord de **CYD-sketch**.

Toetsenbord aansluiten

De CYD V191 gebruikt de MCP23017 via **software-I²C**, op de vierpolige **P4-connector**:

MCP23017	CYD P4	Functie
VCC	Pin 4 — 3V3	Voeding 3,3 V
GND	Pin 1 — GND	Massa
SCL	Pin 2 — GPIO22	I²C-klok
SDA	Pin 3 — GPIO21	I²C-data



De I²C-adressen waarmee V191 de verschillende MCP23017-toetsenborden herkent, zijn gelijk aan die van de 8048S070. **Gebruik het aansluitschema van de 8048S070 niet voor de CYD.** Controleer de pinbezetting van uw specifieke JC2432W328C-revisie.

Geluid

De **8048S070** stuurt digitaal geluid naar de ingebouwde **MAX98357 I²S-versterker**. De **CYD V191** gebruikt daarentegen **GPIO26** met de ESP32-LEDC-periferie, ingesteld op **40 kHz, 8-bit PWM**. De audioroutine geeft de geëmuleerde Atom-PC2-overgangen een korte uitklinktijd. Daardoor kunnen het geluidskarakter en de gebruikte audiohardware verschillen.

De luidsprekeraansluiting moet passen bij de werkelijke CYD-printrevisie; kopieer de I²S-pinnen van de 8048S070 niet.

Verschillen bij software en configuratie

- **8048S070:** sketch `ATOM_6502_8048S070_V191`; SDA GPIO19, SCL GPIO20; MAX98357 I²S-geluid.
- **CYD:** sketch `ATOM_6502_CYD_V191`; SDA GPIO21, SCL GPIO22; GPIO26 PWM-geluid.
- De CYD heeft eigen scherminitialisatie en hardware-instellingen. Volg de configuratie van de CYD-sketch.

Credits en dankwoord

Fake6502

Fake6502 CPU emulator core v1.1 — Mike Chambers, 2011. Fake6502 is de 6502-uitvoerengine die door dit project wordt gebruikt. De broncode is in het publieke domein uitgebracht en vraagt om naamsvermelding. Veel dank aan Mike Chambers voor het beschikbaar stellen ervan.

AtoMMC2

AtoMMC2 is het SD/MMC-filingsysteem dat oorspronkelijk voor de Acorn Atom is ontwikkeld door **Charlie Robson (SirMorris / EightyBits)**, met later werk van **Phill Harvey-Smith** en andere bijdragers uit de

Atom-community. Deze emulator gebruikt de originele AtoMMC-ROM aan de Atom-zijde en emuleert de controllerinterface die deze ROM nodig heeft.

ASBK / Schakelsoft

De ASBK-I- en ASBK-S-software komt voort uit historisch Nederlands Acorn Atom-communitywerk van **Aren Slootweg en Bas Kasteel**. V191 gebruikt ASBK-I v1.03 in bank 1 en een gereconstrueerde ASBK-S-bron waarvan de V24 circulaire zoekfunctie en AtoMMC-vectoruitbreiding het mogelijk maken commando's uit de acht gebankte utility-ROM's in willekeurige volgorde te gebruiken. De originele software en documentatie behouden hun respectieve auteursrechtelijke status.

Acorn Atom

Het Acorn Atom-hardwareontwerp, de ROM-software, BASIC en de originele documentatie zijn het werk van Acorn en hun respectieve auteurs/rechthebbenden. Originele ROM's en commerciële software behouden hun oorspronkelijke auteursrechten.

Schermbibliotheken

De ESP32-S3 8048S070-versie gebruikt **LovyanGFX**. Behoud en volg de licenties van alle third-party-bibliotheken die in de buildomgeving zijn opgenomen of daarvoor nodig zijn.

Community- en archiefbronnen

Originele Atom-handleidingen, softwarearchieven, cassette-images en onderzoek door de community zijn van onschatbare waarde geweest om te begrijpen hoe echte software de machine gebruikt. Third-party-software en documentatie behouden hun eigen licenties en auteursrechten.

Projectstatus

V191 is een stabiele ontwikkelmijlpaal, niet het einde van het project. De emulator biedt nu een compacte zelfstandige Atom-omgeving op de 7-inch ESP32-S3 8048S070 en kan een aanzienlijke hoeveelheid originele software uitvoeren met behoud van de originele 6502-uitvoering, hardware-interfaces en ROM-gedrag.

Op de 8048S070 start V191 bovendien rechtstreeks in de uitgebreide `ACORN ATOM+`-omgeving, waarbij AtoMMC2, acht utility-ROM-banken en residente Schakelsoft direct beschikbaar zijn.

Mogelijk toekomstig werk omvat:

- afronden van minder vaak gebruikte AtoMMC2-functies
- ondersteuning voor random-access AtoMMC2-bestanden
- bredere compatibiliteitstests
- extra Atom-randapparatuur
- verdere compatibiliteitstests voor software met ongebruikelijke toetsenbordtoegang

- verdere audioverfijning wanneer aanvullende software een generiek compatibiliteitsprobleem aantoont